作业1：Data Types, Functions and Conditionals

作者：丁平尖

注意事项

1. 尽早开始！如果在安装或导入包时遇到问题，最好提前发现这些问题，而不是在作业截止前一晚。
2. 确保备份您的工作！推荐学习使用git。
3. 请先将第一个代码框中的学号**修改为自己的学号**，再完成以下作业，并**按顺序运行各个代码框**。

```
import os, uuid, hashlib, platform, getpass

# ===== 1. 填写学号 =====
STUDENT_ID = 20190010  #修改为自己的学号

# ===== 2. 打印 conda 环境 =====
CONDA_ENV = os.environ.get("CONDA_DEFAULT_ENV", "base")
print(f"[Conda 环境] {CONDA_ENV}")

# ===== 3. 机器指纹（绑定当前电脑）=====
MACHINE_FP = hashlib.sha256(
    f"{platform.node()}|{platform.machine()}|{platform.system()}|
{uuid.getnode()}".encode()
).hexdigest()[:16]

# ===== 4. 生成唯一 ID =====
RUN_ID = f"{STUDENT_ID}-{MACHINE_FP}"
print(f"[学号]     {STUDENT_ID}")
print(f"[机器指纹] {MACHINE_FP}")
print(f"[RUN_ID]   {RUN_ID}")
```

作业写作和提交说明

未能遵循这些说明将导致失分。您的作业应该写在一个jupyter笔记本文件中。提交一个名为 **学号_姓名_hwX** 的.ipynb文件，其中X是作业编号。此次作业编号为01.

热身：定义简单函数

在这个问题中，您将练习在Python中定义简单函数。

1. 定义一个名为 `say_hi` 的函数，它不接受任何参数，并在被调用时打印字符串 "Hello, world!"。
2. 定义一个名为 `goat_pad` 的函数，它接受一个字符串作为其唯一参数，并打印该字符串，前后都加上字符串 "goat"。
3. 定义一个名为 `print_n` 的函数，它接受两个参数，一个字符串 `s` 和一个整数 `n`，并打印字符串 `n` 次，每次在单独的一行上。

欧几里得算法

欧几里得算法是一种寻找两个数的最大公约数（GCD）的方法。回想一下，两个数`m`和`n`的最大公约数是同时能整除`m`和`n`的最大数。

1. 实现一个名为`gcd`的函数，它接受两个整数作为其参数并返回它们的GCD。你可以假设两个输入都是整数，所以你不需要在你的函数中包含任何错误检查。

2. 使用你的函数评估以下数字对的GCD：

    1. (2018, 2019)
    2. (1200, 300)
    3. (5040, 60)

3. 如果一个或两个参数是负数，你的函数会做什么？这种行为有意义吗？

近似欧拉数e

自然对数的底数e通常被定义为无限和

$$e = \sum_{k=0}^{\infty} \frac{1}{k!} = 1 + \frac{1}{1} + \frac{1}{2} + \frac{1}{6} + \frac{1}{24} + \dots \tag{1}$$

其中 $k!$ 表示k的阶乘，$k! = k \cdot (k-1) \cdot (k-2) \cdot \ldots \cdot 3 \cdot 2 \cdot 1$，我们约定 $0! = 1$。

1. 欧拉数的一个早期特征，归功于雅各布·伯努利，是作为极限

$$\left(1+\frac{1}{x}\right)^x \quad \text{as} \quad x \to \infty \tag{2}$$

定义一个名为`euler_limit`的函数，它接受一个整数 n 作为参数，并返回一个浮点数，通过上述公式中取 $x = n$ 来近似e。

2. 定义一个名为`euler_infinite_sum`的函数，它接受一个单一的非负整数参数 n，并基于公式（1）中的前 n 项之和返回e的近似值。你的函数应该返回一个浮点数。你可以假设输入将是一个非负整数，所以你不需要在你的函数中包含错误检查。例如，`euler_infinite_sum(4)`应该返回方程1中前四项的和，这样`euler_infinite_sum(4)`返回 $1 + 1 + \frac{1}{2} + \frac{1}{6} \approx 2.667$。注意：公式1中的和从 $k = 0$ 开始计数（即，它是“0-索引”的），而我们的函数从 $n = 1$ 开始计数（即，它是“1-索引”的）。`euler_infinite_sum(1)`应该使用公式（1）中的一个项，这样`euler_infinite_sum(1)`返回 1。类似地，`euler_infinite_sum(0)`应该返回 0。

3. 定义一个名为`euler_approx`的函数，它接受一个单一的参数，一个浮点数 ϵ，并使用公式（1）中的和来获得一个近似e的值，该值在 ϵ 内接近真实值e。提示：使用while循环。注意：你可以使用Python的math模块来获得e的真实值（直到浮点精度）：`math.exp(1)`。

4. 定义一个名为`print_euler_sum_table`的函数，它接受一个单一的正整数 n 作为参数，并打印从`euler_infinite_sum(k)`获得的值，k的范围从1到 n，每行一个。

测试整数属性

在这个问题中，你将得到更多关于条件的练习，并对典型的“编程面试”问题所需的思维方式有一个初步的了解。一个正整数 n 是2的幂，如果 $n = 2^p$ 对于某个整数 $p \geq 0$ 。

1. 编写一个名为`is_power_of_2`的函数，它接受一个正整数作为其唯一的参数，并返回一个布尔值，指示输入是否是2的幂。你可以假设输入是一个正整数。你不能在你的解决方案中使用内置的math.sqrt函数。你应该只需要使用除法和模（%）运算。
2. 将你之前的解决方案推广到一个名为`is_power`的函数，它接受两个正整数作为其参数，b和 n，按那个顺序，并返回一个布尔值。`is_power(b,n)`如果 n 是 b 的幂则返回True，否则返回False。